1.5.1 Example of a consumer contract test

Looking at the consumer-web/consumer-contract.spec.js file of the example GitHub project, one of the example tests is getting a list of movies. A contract test from a consumer perspective looks like the following:

```
const provider = new PactV3({    #1
  consumer: 'WebConsumer',      #1
  provider: 'MoviesAPI',        #1
})                              #1

const EXPECTED_BODY = {                        #2
  id: 1,                                      #2
  name: "My movie",                           #2
  year: 1999                                  #2
}                                             #2

...

provider                                      #2
  .uponReceiving('a request to all movies')  #2
  .withRequest({                              #2
    method: 'GET',                            #2
    path: '/movies',                          #2
  })                                          #2
  .willRespondWith({                          #2
    status: 200,                              #2
    body: eachLike(EXPECTED_BODY)             #2
  })                                          #2
})                                             #2

await provider.executeTest(async mockProvider) => {     #3
  const movies = await fetchMovies(mockProvider.url)    #3
  expect(movies[0]).toEqual(EXPECTED_BODY);             #3
})                                                      #3
```

#1 Set up the mock provider.
#2 Set up expectations from the provider.
#3 Run the consumer test against the mock provider. The mock provider validates the test, and, if successful, the contract is generated.

The key thing to note here is that the consumer doesn’t interact with the real provider but with a mock provider instead, which is provided by the contract testing tool. We’ll further explain the process of writing a consumer contract test in chapter 4. Once the test is run, a contract, similar to the one shown here, is generated by the contract testing tool, which contains the expectations the consumer has from the provider: